

Manual Técnico del Sistema Web MYVETPET Veterinaria

Víctor Daniel Tepáz Morente, 202202119,
Alejandra Magali López Miranda, 201600085

Resumen—Se desarrolló una plataforma web profesional para la clínica veterinaria MYVETPET utilizando Django, Python y PostgreSQL, desplegada mediante Docker y alojada en AWS EC2. El sistema permite gestionar usuarios, mascotas, citas, servicios veterinarios y generar documentos PDF. Se implementó una arquitectura modular basada en el patrón MVT de Django, integrando autenticación segura, validación de datos, panel administrativo y un flujo de reservas intuitivo. Este documento presenta el manual técnico completo del sistema, incluyendo arquitectura, instalación, despliegue, estructura interna y funcionamiento del sistema.

I. INTRODUCCIÓN

El sistema web MYVETPET fue desarrollado para digitalizar y optimizar procesos administrativos y clínicos de una veterinaria moderna. Utiliza tecnologías como Django, Python, PostgreSQL y Docker, permitiendo una administración eficiente de usuarios, mascotas, servicios y citas en línea. El proyecto incorpora un diseño modular, seguridad basada en roles, envío de correos, generación de archivos PDF para reportes y despliegue en la nube mediante AWS. Este manual técnico describe su arquitectura, funcionamiento interno y procedimiento de instalación.

II. OBJETIVOS

II-A. General

Diseñar e implementar un sistema web escalable y seguro para la gestión integral de una clínica veterinaria, empleando tecnologías modernas basadas en Python.

II-B. Específicos

- * Implementar autenticación segura para clientes y personal administrativo.
- * Integrar una base de datos PostgreSQL para almacenar información de usuarios, mascotas, citas y servicios.
- * Desarrollar un sistema de citas eficiente y accesible desde cualquier dispositivo.
- * Generar documentos PDF utilizando HTML y xhtml2pdf.
- * Desplegar el sistema en la nube mediante Docker y AWS EC2.

III. ARQUITECTURA DEL SISTEMA

El sistema está compuesto por los siguientes módulos principales:

- **Frontend:** Django Templates, HTML5, CSS3.
- **Backend:** Django 5, Python 3.12.

- **Base de Datos:** PostgreSQL.
- **Contenedores:** Docker y Docker Compose.
- **Despliegue:** Servidor AWS EC2.

IV. TECNOLOGÍAS IMPLEMENTADAS

IV-A. Django

Framework de alto nivel utilizado para la estructura del backend, manejo del ORM, enrutamiento y seguridad.

IV-B. PostgreSQL

Base de datos relacional que almacena la información crítica del sistema.

IV-C. Docker

El proyecto se ejecuta mediante contenedores:

- Contenedor Web (Django)
- Contenedor DB (PostgreSQL)

IV-D. AWS

El sistema se despliega en una instancia EC2 con Ubuntu 22.04.

V. REQUERIMIENTOS DEL SISTEMA

V-A. Software

- Python 3.12
- Django 5.2.7
- PostgreSQL 15
- Docker 24+
- Docker Compose
- Ubuntu 22.04 (para despliegue)

V-B. Dependencias

Archivo `requirements.txt` del proyecto:

```
asgiref==3.10.0
Django==5.2.7
pillow==10.2.0
psycopg2-binary==2.9.11
pytz==2024.1
sqlparse==0.5.3
xhtml2pdf==0.2.8
reportlab==3.6.12
```

VI. INSTALACIÓN DEL SISTEMA

VI-A. Clonar repositorio

```
git clone https://github.com/AlejandraMLM/980-Proyecto-veterinaria
```

VI-B. Construir contenedores

```
sudo docker compose build --no-cache
sudo docker compose up -d
```

VI-C. Migraciones

```
sudo docker exec -it veterinaria_web_container bash
python3 manage.py migrate
python3 manage.py createsuperuser
```

VII. ESTRUCTURA DEL PROYECTO

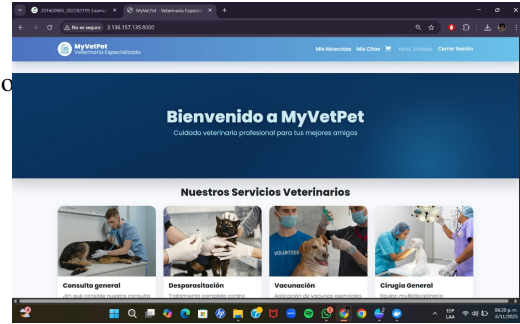
```
veterinaria /
├── acceso /
├── administrador /
├── citas /
├── mascotas /
├── servicios /
├── veterinario /
├── templates /
├── static /
├── media /
├── Dockerfile
├── docker-compose.yml
└── manage.py
```

VII-A. Modelos

```
1 from django.contrib.auth.models import AbstractUser
2 from django.db import models
3 from django.conf import settings
4 from django.utils import timezone
5
6 class Cliente(AbstractUser):
7     dni = models.CharField(max_length=13, unique=True)
8     telefono = models.CharField(max_length=15)
9     direccion = models.TextField()
10
11     def __str__(self):
12         return self.username
13
14 class Mascota(models.Model):
15
16 class Servicio(models.Model):
17
18 class Cita(models.Model):
```

Modelos implementados, 2025.

VII-B. Views



Estructura de vistas del proyecto.

VIII. SISTEMA DE GENERACIÓN DE PDFS

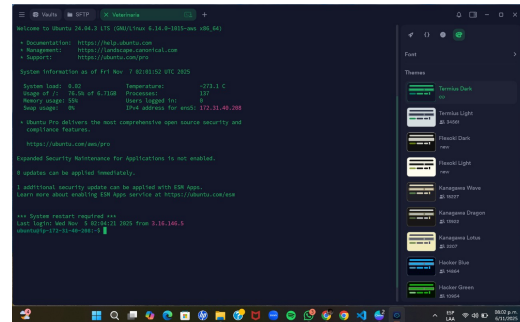
El sistema genera boletas, reportes y comprobantes utilizando HTML convertido a PDF mediante la librería xhtml2pdf.

Limitaciones:

- No soporta CSS avanzado.
- Requiere compatibilidad con la versión de reportlab instalada.

IX. DESPLIEGUE EN AWS EC2

IX-A. Conexión mediante Termius



Conexión SSH desde Termius.

IX-B. Servidor corriendo

Sistema operativo desde EC2.

X. RESULTADOS

- Sistema web completamente funcional.
- Gestión de citas, mascotas y usuarios implementada.
- Generación de archivos PDF operativa.
- Despliegue estable en AWS.
- Contenedores Docker funcionando correctamente.

XI. CONCLUSIONES

- * Django permitió un desarrollo modular, seguro y escalable.
- * PostgreSQL garantizó integridad, consistencia y rendimiento.
- * Docker facilitó el despliegue consistente entre entornos.

- * AWS EC2 proporcionó disponibilidad y acceso global al sistema.
- * Termius simplificó las conexiones SSH y la administración del servidor.

XII. REPOSITORIO EN GITHUB

<https://github.com/AlejandraMLM/980-Proyectos/tree/main/veterinaria>

REFERENCIAS

- [1] Django Software Foundation. *Django Documentation*. Disponible en: <https://docs.djangoproject.com/>
- [2] PostgreSQL Global Development Group. *PostgreSQL Documentation*. Disponible en: <https://www.postgresql.org/docs/>
- [3] Docker Inc. *Docker Documentation*. Disponible en: <https://docs.docker.com/>
- [4] xhtml2pdf Team. *xhtml2pdf Documentation*. Disponible en: <https://xhtml2pdf.readthedocs.io/>